

Enhancing Symbolic Execution with Self-Configuring Parameters

Minjong Kim
Sungkyunkwan University
Suwon, Republic of Korea
minjong.kim@skku.edu

Sooyoung Cha*
Sungkyunkwan University
Suwon, Republic of Korea
sooyoung.cha@skku.edu

Abstract

We present PARASUIT, a self-configuring technique that enhances symbolic execution by autonomously adjusting its parameters tailored to each program under test. Modern symbolic execution tools are typically equipped with various external parameters to effectively test real-world programs. However, the need for users to fine-tune a multitude of parameters for optimal testing outcomes makes these tools harder to use and limits their potential benefits. Despite recent efforts to improve this tuning process, existing techniques are not self-configuring; they cannot dynamically identify which parameters to tune for each target program, and for each manually selected parameter, they sample a value from a fixed, user-defined set of candidate values that is specific to that parameter and remains unchanged across programs. The goal of this paper is to automatically configure symbolic execution parameters from scratch for each program. To this end, PARASUIT begins by automatically identifying all available parameters in the symbolic execution tool and evaluating each parameter's impact through interactions with the tool. It then applies a specialized algorithm to iteratively select promising parameters, construct sampling spaces for each, and update their sampling probabilities based on data accumulated from symbolic execution runs using sampled parameter values. We implemented PARASUIT on KLEE and assessed it across 12 open-source C programs. The results demonstrate that PARASUIT significantly outperforms the state-of-the-art method without self-configuring parameters, achieving an average of 26% higher branch coverage. Remarkably, PARASUIT identified 11 unique bugs, four of which were exclusively discovered by PARASUIT.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Machine learning*.

Keywords

Symbolic execution, Parameter configuring, Software testing

ACM Reference Format:

Minjong Kim and Sooyoung Cha. 2026. Enhancing Symbolic Execution with Self-Configuring Parameters. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3773264>

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICSE '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3773264>

1 Introduction

Symbolic execution [15, 16, 38, 73] is undoubtedly one of the prominent software testing methods, actively researched in both academia and industry [14, 39, 40]. Its primary goal is to efficiently generate valuable test-cases that cover hard-to-reach execution paths in the target program or reveal hidden bugs within a time budget. To achieve this, symbolic execution translates program inputs into symbolic variables, runs the target program with those variables, accumulates symbolic branch conditions, and generates test-cases that satisfy these conditions using an SMT solver [30, 37]. However, due to the need to gather symbolic branch conditions and generate test-cases via the SMT solver, symbolic execution faces two well-known open challenges: the path-explosion problem and the high cost of constraint solving. In response to these challenges, various techniques have been proposed, including search strategy [19, 44, 57, 93], path-pruning [8, 22, 90, 91], constraints solving [54, 60, 78, 81], and state-merging [3, 55, 74, 82]. As a result, the effectiveness of modern symbolic execution tools incorporating these techniques has seen marked improvement.

Yet, despite their notable performance enhancements, it has paradoxically become more challenging for users to utilize symbolic execution tools effectively. This challenge stems from the need for users to determine the "parameter values" that influence the effectiveness of these tools to achieve the desired testing outcomes. Specifically, given a target program, users choose which parameters to use from the total parameters provided by a symbolic execution tool, and then assign a suitable value to each selected parameter. Unfortunately, the number of adjustable parameters in practical symbolic execution tools, like KLEE [14], exceeds a hundred, including boolean, string, and integer types. Thus, adjusting these parameter values for testing each program imposes a considerable burden on users. This burden often leads users to settle for the default parameter settings offered by symbolic execution tools. For example, many previous works [22, 23, 35, 87] have depended on the parameter settings in Figure 1-(a) provided by the original authors of KLEE. Other works [54, 57, 69, 90] have manually adjusted only a handful of parameters to effectively test the program.

Although recent techniques have aimed to effectively tune either a specific parameter [18, 44] or multiple parameters [20], given the importance of parameter tuning for symbolic execution tools, they are not self-configuring: they cannot dynamically select which parameters to tune for each target program, and even for each manually chosen parameter, they sample a value from a fixed, pre-defined set of candidate values that is specific to that parameter and remains unchanged across programs. For example, SymTuner [20] relies on manually defined parameter spaces to adjust multiple parameters of symbolic execution. Figure 1-(b) illustrates SymTuner's hand-crafted parameter spaces, constructed by selecting 20 parameters from over a hundred available for tuning and defining 5 to

```
$ klee --simplify-sym-indices=true --max-memory=1000
--max-static-fork-pct=1.0 --search=random-path
... (24 more parameters)
--max-solver-time=30s --sym-args 0 1 10 --sym-files 1 8
```

(a) Parameter values of KLEE used in [66]

```
"space": {
  "-simplify-sym-indices": [True, False],
  "-max-memory": [500, 1000, 1500, 2000, 2500],
  "-max-static-fork-pct": [0.25, 0.5, 1, 2, 4],
  "-search": ["random-path", ..., "dfs"],
  ... (13 more spaces)
  "-max-solver-time": ["10s", "20s", "30s", "40s", "50s"],
  "-sym-arg": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
  "-sym-files 1": [4, 8, 12, 16, 20]
}
```

(b) Parameter spaces of KLEE used in [20]

Figure 1: Parameter settings for symbolic execution tools

10 possible candidate values for each. As we show in Section 4.5, effective parameters and their value ranges vary greatly depending on the target program. Therefore, these non-self-configuring techniques fail to realize the true benefits of parameter tuning for symbolic execution, highlighting the importance of program-aware parameter tuning. Without such self-configuration, for each program, users must manually select relevant parameters and assign plausible values for each chosen parameter, adding complexity.

We present PARASUIT, a new self-configuring approach that autonomously selects suitable parameters and determines their values in a program-aware way from scratch, thereby maximizing the performance of symbolic execution. To eliminate the need for user intervention, PARASUIT begins by automatically extracting all available parameters from the abstract syntax tree (AST) of the symbolic execution tool's source code and assessing the potential impact of each parameter through interactions with the tool. Once this extraction stage is complete, PARASUIT enters two iterative stages: parameter selection and value sampling. In the selection stage, PARASUIT assigns a score to each parameter using a customized scoring function that captures both its ability to reach rarely explored branches and its synergy with others, selecting the most promising ones for tuning. In the sampling stage, PARASUIT collects data by systematically exploring and evaluating diverse values for the selected parameters. Based on the collected data, PARASUIT constructs sampling spaces and assigns their probabilities for each selected parameter. It then samples parameter values according to these probabilities and runs the symbolic execution tool on the target program using these values. The results from each run are added to the accumulated dataset. As these two stages repeat, PARASUIT continually refines the dataset, enabling it to provide increasingly effective parameter values tailored to the target program.

Our experimental results demonstrate PARASUIT's ability to significantly enhance the performance of symbolic execution tools when integrated. This improvement is evident in both branch coverage and the identification of unique bugs. We built PARASUIT atop KLEE [14], a well-engineered symbolic execution tool replete

with diverse parameters. For our evaluation, we used 12 open-source C programs and compared PARASUIT against two baselines: KLEE with manually tuned parameter values (as shown in Figure 1-(a)) and the state-of-the-art SYMTUNER [20], a non-self-configuring method that utilizes user-defined spaces (as illustrated in Figure 1-(b)). In terms of branch coverage, PARASUIT outperformed the non-self-configuring method by 25.8% and the manual tuning approach by a substantial 97.4%. In the realm of bug detection, PARASUIT identified 11 unique bugs, outperforming the next best method, which found only seven. Furthermore, the results highlight the efficiency of PARASUIT through comparisons with three generic value sampling algorithms and demonstrate its generality by applying it to another dynamic symbolic execution tool, CREST [29].

Contributions Our contributions are summarized as follows:

- To our knowledge, PARASUIT is the first self-configuring technique for tuning symbolic execution parameters from scratch in a program-aware manner without any user intervention.
- Our key contribution is to develop a customized algorithm that, without any prior knowledge, automatically identifies program-specific effective parameters and derives suitable values through iterative refinement informed by symbolic execution feedback.
- We demonstrate the effectiveness of PARASUIT by comparing it against both a manual approach and a state-of-the-art non-self-configuring approach across 12 open-source C programs. We make the implementation of PARASUIT¹ publicly accessible: <https://github.com/unknownicse2026/parasuit>

2 Preliminaries

2.1 Symbolic Execution and Its Parameters

Symbolic execution aims to generate test-cases that cover diverse execution paths of the target program. To achieve this, it repeatedly chooses, executes, and updates a so-called "state", all while preserving a set of states during a given time budget. Each state consists of three elements: the next statement (*stmt*) to be executed, a symbolic store (*store*) which maps program inputs to symbolic variables, and a path condition (Φ), a conjunction of symbolic branch conditions.

Initially, symbolic execution initializes a set of states with a singleton set containing an initial state ($stmt_0, store_0, \Phi_0$). Here, $stmt_0$ represents the first statement of the program, $store_0$ indicates the mapping information from program inputs to symbolic variables (e.g., $\text{int } x \rightarrow \alpha$), and Φ_0 is set to *true*. Symbolic execution then iteratively chooses a state ($stmt, store, \Phi$) from the set, executes the next statement ($stmt'$) for the chosen state, and updates the state based on the type of *stmt*. For instance, if *stmt* is a conditional statement with condition φ , it checks whether the two path-conditions— $(\Phi \wedge \varphi)$ for the true branch and $(\Phi \wedge \neg\varphi)$ for the false branch—are satisfiable using the SMT solver. If both are feasible, the state is forked into two distinct states $s_1 (stmt_{\text{true}}, store', \Phi \wedge \varphi)$ and $s_2 (stmt_{\text{false}}, store', \Phi \wedge \neg\varphi)$. If the statement is a halt statement, a test-case is generated based on the accumulated path-condition in the state. In summary, performing symbolic execution entails repeatedly executing the statements of the target program from its entry point (e.g., main function), collecting branch conditions

¹PARASUIT is a combination of **Suitable** and **Parameters**.

associated with symbolic variables, and producing a test-case based on the conjunction of conditions accumulated up to its endpoint.

Practical symbolic execution tools typically come equipped with numerous parameters to effectively test real-world programs, requiring users to decide on these parameter values. In this work, we define parameters as all adjustable factors influencing the performance of symbolic execution. For instance, a well-engineered symbolic execution tool, KLEE [14], incorporates a total of 148. Consider some parameters associated with a state (*stmt*, *store*, Φ) as shown in Figure 1-(a). The "max-solver-time" parameter, linked with the first element, *stmt*, limits the maximum time the solver spends finding a solution to a single query. The "sym-args" parameter, tied to the second element, *store*, determines the number and length of symbolically generated arguments. The "search" parameter pertains to the state-selection process, guiding which states to explore first. The "max-memory", addressing the state-explosion problem, sets a cap on the number of states that can be retained during symbolic execution. Notably, KLEE encompasses 148 parameters, including 27 of integer types, 6 of float types, 24 of string types, and 91 of boolean types. Therefore, to optimally test a target program in terms of code coverage and bug-finding, users must identify useful combinations of these parameter values.

2.2 Existing Parameter Tuning Approaches

Current primary approaches for tuning parameter values in symbolic execution tools share a key limitation: these approaches are not self-configuring with respect to the program under test, as they use fixed parameter choices and select values from parameter-specific, predefined sets that do not vary with the target program.

Manual Tuning Approach. The most prevalent approach [22, 35, 44, 54] involves users manually tuning parameter values to effectively test the target program. Specifically, users first decide which parameters to adjust from the complete set provided by the symbolic execution tool. Then, they determine an appropriate value for each chosen parameter and execute the symbolic execution tool on the target program using these values. Since configuring each parameter individually poses challenges even for domain experts, many prior symbolic execution works have utilized the parameter values depicted in Figure 1-(a) without any modifications [22, 35, 81, 82]. Alternatively, some works have manually adjusted only a handful of parameters (e.g., "sym-args" and "sym-files") suitable for each program while retaining the remaining parameter values from Figure 1-(a) unchanged [11, 44, 54, 80].

User-configured Tuning Approach. Although a state-of-the-art approach [20] has recently attempted to automate the tuning process for symbolic execution parameters, it remains user-configured because it requires users to manually define parameter spaces. Each parameter space is defined as a pair of a parameter and its candidate values, e.g., ("max-memory", {500, 1000, ..., 2500}), as illustrated in Figure 1-(b). Furthermore, even with these hand-crafted spaces, it is not self-configuring, as it always tunes the same set of parameters and selects values from fixed, parameter-specific candidate sets regardless of the target program. Note that manually defining these parameter spaces is as challenging for users as directly selecting parameter values. Users must first identify relevant parameters (e.g.,

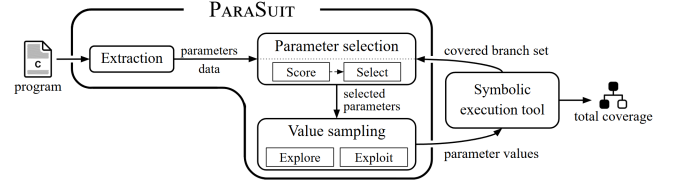


Figure 2: Overview of PARASUIT.

20 parameters as in [20]) from the full set of 148 available parameters, and then determine suitable values for each selected parameter (e.g., "sym-arg" = {1, 2, ..., 10}). In other words, the stages of manually selecting parameters and constructing candidate values for each make this approach non-self-configuring, posing a critical hurdle to maximizing the benefits of external parameter tuning.

Goal. The objective of this paper is to automate program-aware tuning of external parameters for symbolic execution. Specifically, for each program, we autonomously select relevant parameters from the complete set and determine their appropriate values without any prior knowledge. We can formally define our goal of maximizing the benefits of tuning external parameters by using the following symbolic execution procedure (EXECUTE), where it takes target program (*pgm*), time budget (*time*), and parameter values (*PV*) as inputs and returns the branch set *B* covered until *time* expires. Using this procedure, our goal is to iteratively identify sets of parameters and values such that the union of symbolic execution outcomes

$$\bigcup_{i=1}^n \text{EXECUTE}(\text{pgm}, \text{time}_s, \text{PV}_i)$$

is maximized within the total budget (*time*), where *time_s* denotes the smaller, individual time allocations within *time*. Ultimately, our novel approach, PARASUIT, eliminates the need for users to manually adjust parameters while achieving the highest branch coverage through fully self-configured parameters.

3 Our Approach

3.1 Overview

Figure 2 provides an overview of PARASUIT, which operates in three main stages: Extraction, Selection, and Sampling. Initially, PARASUIT requires only a target program as input, meaning that users do not need to make any additional effort to tune symbolic execution parameters in a program-aware manner. The first stage, Extraction, gathers the baseline data necessary for parameter selection. In this stage, PARASUIT automatically identifies all available parameters and assesses the potential impact of each parameter through interactions with the symbolic execution tool.

Once the Extraction stage is complete, PARASUIT iteratively executes the subsequent two stages—Selection and Sampling—until the allocated total budget expires. In the Selection stage, PARASUIT calculates a score for each parameter using a customized scoring function and selects the most promising parameters for tuning based on these scores. Intuitively, parameters receive higher scores if they tend to cover many of the rarely reached branches and work synergistically with others to increase coverage. In the Sampling stage, PARASUIT collects data by systematically exploring and evaluating diverse values for the selected parameters. Based on the

Algorithm 1 PARASUIT

Input: Program (pgm), time budget ($time$)
Output: A total set of covered branches ($UnionB$)

```

1:  $\langle D, BaseD, TotalB, time_s \rangle \leftarrow \langle \emptyset, \emptyset, \emptyset, \eta_{time} \rangle$   $\triangleright \eta_{time} = 120$ 
2:
3: /* Step 1. Extraction stage */
4:  $P \leftarrow \text{Initialize}()$   $\triangleright (p, v_d, V) \in P$ 
5: for each  $(p, v_d, V) \in P$  do
6:    $PV_d \leftarrow \{(p', v_d) \mid (p', v_d, \_) \in P \wedge (p' \neq p)\}$ 
7:   for  $v$  in  $V \setminus \{v_d\}$  do
8:      $PV \leftarrow PV_d \cup \{(p, v)\}$ 
9:      $B \leftarrow \text{EXECUTE}(pgm, time_s, PV)$   $\triangleright B = \text{covered branches}$ 
10:     $BaseD \leftarrow BaseD \cup \{(p, v, B)\}$ 
11:     $TotalB \leftarrow TotalB \cup \{B\}$ 
12:
13: repeat
14:   /* Step 2. Parameter selection stage */
15:    $BaseS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_) \in BaseD\}$ 
16:    $ParamS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_) \in D\}$ 
17:    $SelectP \leftarrow \text{Select}(ParamS, BaseS)$ 
18:
19:   /* Step 3. Value sampling stage */
20:    $PV \leftarrow \text{SAMPLE}(SelectP, D)$ 
21:    $B \leftarrow \text{EXECUTE}(pgm, time_s, PV)$ 
22:    $TotalB \leftarrow TotalB \cup \{B\}$ 
23:    $D \leftarrow D \cup \{(p, v, B) \mid (p, v) \in PV\}$ 
24: until  $time$  expires
25: return  $UnionB$   $\triangleright UnionB = \bigcup_{B \in TotalB} B$ 

```

data, PARASUIT constructs sampling spaces and assigns their corresponding probabilities for each selected parameter. Finally, it then samples parameter values according to these probabilities and executes the symbolic execution tool on the target program using the sampled values. The results from these executions are accumulated as additional data. Once the total budget is consumed, PARASUIT returns the cumulative set of covered branches across all stages. As the Selection and Sampling stages repeat, PARASUIT continually refines the data, allowing it to provide progressively well-suited parameter values for the target program. This iterative process ultimately maximizes the performance of symbolic execution.

3.2 PARASUIT

Algorithm 1 offers a detailed description of how PARASUIT iteratively selects promising parameters and samples their values tailored for the target program, requiring no user effort. PARASUIT accepts only two inputs: the program (pgm) to be tested and the total time budget ($time$). Once $time$ is exhausted, Algorithm 1 outputs the total set of covered branches ($UnionB$). At line 1, it initializes three sets— D , $BaseD$ and $TotalB$ —as empty sets. It also sets a small time budget ($time_s$) to a hyper-parameter value (η_{time}). Since PARASUIT aims to find suitable parameters and their values for the target program without prior knowledge, it inevitably needs to run the symbolic execution tool multiple times, each with newly defined parameter values and a small time budget. This iterative process enables PARASUIT to explore diverse parameter settings based on accumulated data. In our experiments, the initial small time budget (η_{time}) was set to 120 seconds by trial and error.

Extract. At lines 4–11, the Extraction stage aims to construct the dataset ($BaseD$), which serves as the criterion for assessing the potential of each parameter during parameter selection. To gather data, Algorithm 1 first identifies all available parameters by executing the symbolic execution tool with the "help" command.

For example, running the well-known symbolic execution tool, KLEE [14], with the command `./klee --help` extracts all available 148 parameters, such as "search" and "max-memory":

```

$ ./klee --help
-search - Specify the search heuristic
-max-memory - Refuse to fork when hit the memory (MB)
...

```

For each identified parameter p , Algorithm 1 extracts its default value (v_d) and possible values (V) from the abstract syntax tree (AST) of the symbolic execution tool's source code, constructing the set P of parameters, $(p, v_d, V) \in P$, where V is used to evaluate the potential impact of p . Specifically, using the AST generated by Tree-sitter [7], PARASUIT identifies the code block where each parameter is defined. For example, for the "search" parameter in KLEE, the following block is extracted from the source code:

```

1  cl::list<Searcher::SearchType> Search(
2  "search",
3  cl::desc("Specify search heuristic"),
4  cl::values(
5      clEnumValN(Searcher::Random, "random-path"),
6      clEnumValN(Searcher::DFS, "dfs"),
7      ... (9 more candidates) ),
8  cl::init("random-path"));

```

From this block, we capture the default value (v_d) and the possible values (V) for the "search" parameter as:

$v_d = \text{random-path}, V = \{\text{random-path}, \text{dfs}, \dots\}$

For string or boolean parameters, all possible values can be directly extracted. However, for integer or float parameters, the possible values are infinite. Thus, we heuristically define the set V , which consists of a value near the default:

$V = \{v_d \times r\}, (0.5 \leq r \leq 1.5) \wedge (r \neq 1).$

This approach allows for a more accurate estimation of the potential of each parameter to enhance symbolic execution by utilizing values close to the default, rather than choosing randomly from a vast pool of candidate values. For instance, the integer parameter max-memory with a default of 2GB can be represented in P as:

$(\text{"max-memory"}, 2GB, \{2.4GB\}).$

Using this extraction method, PARASUIT extracts the parameters of symbolic execution tools more accurately and efficiently than other existing techniques [33, 71]. Additionally, Algorithm 1 excludes parameters whose names contain specific keywords (e.g., "log", "version") that are unlikely to impact symbolic execution performance. For example, out of 148 total parameters in the symbolic execution tool KLEE, keyword filtering reduces the number of parameters to 90.

After constructing a triplet (p, v_d, V) for each parameter p , the algorithm evaluates the impact of p through interactions with the symbolic execution tool (lines 5–11). Specifically, for each p in P , it first constructs PV_d , a parameter setting where all parameters except p are assigned their default values (line 6). Then, it sets PV as the union of PV_d and $\{(p, v)\}$ at line 8, where v is an element of

$V \setminus \{v_d\}$, i.e., the set of possible values excluding the default. At line 9, the Run procedure executes with PV and returns the set B of covered branches. Intuitively, the set B estimates the potential impact of the parameter itself since it is covered by modifying only one specific parameter value while keeping all others at their default values. At line 10, Algorithm 1 then adds the triplet (p, v, B) into dataset $BaseD$. At line 11, instead of directly adding the set B to $TotalB$, the algorithm deliberately adds a singleton set of B to $TotalB$. This method of defining $TotalB$ helps measure the frequency at which each branch is covered, and this frequency is later used to calculate the score. After completing the Extraction stage, Algorithm 1 enters an iterative process of selecting parameters (Selection stage) and sampling values for the selected ones (Sample stage).

3.3 Parameter Selection

The goal of the Selection stage is to choose from all available parameters promising ones that are essential for effectively testing the target program (lines 15–17). Intuitively, we define promising parameters as those that (1) reach many branches, (2) cover infrequently reached branches, and (3) produce synergy when combined with other parameters. To achieve this, Algorithm 1 calculates two distinct scores from two sets, $BaseD$ and D , using a customized scoring function (lines 15–16). $BaseD$ quantifies the effect of tuning a parameter while keeping all others at their default values. In contrast, D measures its performance when tuned simultaneously with others during the iterative process. After calculating scores using the two sets, the Select function determines which parameters to tune based on their scores (line 17).

Specifically, at lines 15–16, the Score function evaluates each parameter p based on two coverage-related aspects: (1) the number of branches covered when adjusting p , and (2) its ability to reach branches that are rarely covered by other parameters. To formally define the Score function, we introduce the Frequency function as:

$$\text{Frequency}(b, TotalB) = |\{B \mid (b \in B) \wedge (B \in TotalB)\}|$$

This function counts the number of times that a branch b appears in the accumulated set $TotalB$ of covered branches. Recall that Algorithm 1 adds a singleton set containing B to $TotalB$ (line 11) to track the frequency of each covered branch. For instance, consider the set $TotalB$ containing two sets of covered branches:

$$TotalB = \{\{b_1, b_2, b_3\}, \{b_1, b_4\}\}$$

In this set, the frequency of b_1 — $\text{Frequency}(b_1, TotalB)$ —is 2 while the frequency of b_2 is 1. Leveraging the Frequency function, we define the Score function as:

$$\text{Score}(B, TotalB) = \sum_{b \in B} \frac{1}{\text{Frequency}(b, TotalB)} \quad (1)$$

Intuitively, this function assigns a higher score when the set B is larger and each branch b in B has a lower occurrence frequency.

At line 17, the Select function first calculates the baseline score (bs_p) of each parameter p when tuned solely as follows:

$$bs_p = \sum_{s' \in S} \frac{s'}{|S|}, \quad S = \{s' \mid (p = p') \wedge (p', s') \in BaseS\}.$$

Subsequently, the function compares the score of each parameter p accumulated in $ParamS$ with its standalone score bs_p . If the score

is lower than bs_p , it is set to zero to penalize the parameters that degrade performance when combined with others:

$$S_p = \{s' \mid (p = p') \wedge (p', s') \in ParamS\}, \quad s' = \begin{cases} s & \text{if } s \geq bs_p \\ 0 & \text{otherwise} \end{cases}.$$

The intuition behind this penalty is that if p performs worse when tuned simultaneously with others compared to tuning it alone, then p likely does not contribute positively in combination. This penalty guides the process away from selecting such parameters. Finally, it computes the average score for p , represented by $\sum_{s' \in S_p} \frac{s'}{|S_p|}$.

After scoring all parameters, the Select function decides whether to select each parameter based on its normalized score. For example, if the score of a parameter p is 0.3, it is included in the set of selected parameters ($SelectP$) with probability 0.3 and excluded with probability 0.7. This process is applied to all parameters, ultimately returning the set $SelectP$. Now, we proceed to the second iterative stage: assigning an appropriate value for each chosen parameter.

3.4 Value Sampling

In this stage, Algorithm 1 iteratively samples an appropriate value for each parameter in $SelectP$ using the SAMPLE procedure (line 20), constructing the set of parameter values PV . Algorithm 2 provides details on how it determines a value for each chosen parameter. It takes two inputs—a set of selected parameters ($SelectP$) and accumulated data (D)—and returns PV , which pairs each parameter with a sampled value. At line 2, it initializes PV as an empty set.

At lines 3–9, for each parameter p , Algorithm 2 selects one of two strategies—Explore or Exploit—based on the accumulated data D , and samples a corresponding value using the chosen strategy. At line 4, the algorithm first extracts the set $ValueS$ from D , where each element is a pair of a used value and its score; the score is calculated using the Score function in Equation (1). The algorithm then clusters the elements in the set $ValueS$ based on their scores:

$$\text{Cluster}(ValueS) = \{C_1, C_2, \dots, C_n\}$$

where each C_i is a subset of $ValueS$ with similar scores, indicating similar performance impacts. For clustering, we use mean shift clustering [25], a well-known non-parametric method that identifies clusters without requiring the number of clusters in advance. After constructing the set C of clusters, Algorithm 2 computes the silhouette coefficient [72] to evaluate the clustering quality (line 6). If the coefficient is below η_{clust} , indicating poorly separated clusters or insufficient data, the algorithm applies the Explore strategy (line 7). Otherwise, it proceeds with the Exploit strategy (line 9). After applying the same decision-making for each parameter in $SelectP$, Algorithm 2 returns the set of newly sampled parameter values. We set the initial value of η_{clust} to 0.7 by trial and error.

Explore. The Explore strategy aims to investigate diverse values for the parameter p to encourage the formation of well-separated clusters with high silhouette coefficients. To do so, it constructs a sampling space SP_p and randomly samples a value from the space. Specifically, SP_p is built around v_{rand} , a randomly chosen value from $ValueS$. Using v_{rand} , SP_p is then defined as:

$$SP_p = 0.5 \times v_{\text{rand}} \leq v' \leq 1.5 \times v_{\text{rand}}.$$

Algorithm 2 Value Sampling Stage

Input: Selected parameters (*SelectP*), data (*D*)
Output: Parameter values (*PV*)

```

1: procedure SAMPLE(SelectP, D)
2: PV  $\leftarrow \emptyset$ 
3: for each p  $\in$  SelectP do
4:   ValueS  $\leftarrow \{(v, \text{Score}(B', \text{TotalB})) \mid (p = p') \wedge (p', v', B') \in D\}$ 
5:   C  $\leftarrow \text{Cluster}(\text{ValueS})$   $\triangleright C = \{C_1, C_2, \dots, C_n\}; C_i \subseteq \text{ValueS}$ 
6:   if  $\text{Silhouette}(C) \leq \eta_{\text{clust}}$  then
7:     PV  $\leftarrow PV \cup (p, \text{Explore}(\text{ValueS}))$   $\triangleright \eta_{\text{clust}} = 0.7$ 
8:   else
9:     PV  $\leftarrow PV \cup (p, \text{Exploit}(C))$   $\triangleright C = \{C_1, C_2, \dots, C_n\}$ 
10: return PV

```

The Explore strategy randomly samples a new value v' from SP_p and adds (p, v') to the set *PV* of parameter values at line 7. For boolean or string-type parameters, which do not have a numeric range, the strategy instead samples v' randomly from the set of all possible values. For example, for a boolean parameter p_b , either (p_b, true) or (p_b, false) is randomly added to *PV*. This process is repeated for parameters assigned to the Explore strategy, which indicates the need for more data to guide efficient value sampling.

Exploit. The Exploit strategy in Algorithm 2 focuses on leveraging the accumulated data for the selected parameter p to sample a new value effectively (line 9). To achieve this, the strategy first constructs a new sampling space SP_p using the set *C* of well-formed clusters generated at line 5, such as $C = \{C_1, C_2, \dots, C_n\}$, where each cluster C_i consists of pairs of previously used values with similar scores. The sampling space SP_p is defined as:

$$SP_p = \{(C_1, \mathcal{P}_1), (C_2, \mathcal{P}_2), \dots, (C_n, \mathcal{P}_n)\}$$

Each element in SP_p represents a cluster C_i and its associated probability $\mathcal{P}_i$, which indicates the likelihood of sampling a new value from C_i . For example, consider the set of clusters $C = \{C_1, C_2\}$ associated with the max-memory parameter:

$$C = \{(2GB, 28), (3GB, 26)\}, \{(0.5GB, 8), (0.1GB, 6)\} \quad (2)$$

Here, $C_1 = \{(2GB, 28), (3GB, 26)\}$ and $C_2 = \{(0.5GB, 8), (0.1GB, 6)\}$. The sampling probability of each cluster C_i is given by dividing the mean score of C_i by the sum of the mean scores of all clusters in *C*:

$$\mathcal{P}(C_i) = \frac{\text{mean}(\{s \mid (_, s) \in C_i\})}{\sum_{1 \leq k \leq |C|} \text{mean}(\{s \mid (_, s) \in C_k\})}$$

This formula implies that clusters with higher average scores are more likely to be selected. For example, using the set *C* from Equation (2), we can calculate the probability for the first cluster C_1 :

$$\mathcal{P}(C_1) = \frac{\frac{28+26}{2}}{\left(\frac{28+26}{2}\right) + \left(\frac{8+6}{2}\right)} = \frac{27}{34}$$

After constructing SP_p , the Exploit strategy selects a cluster C_i from *C* based on the sampling probabilities and then samples a new value v' from that cluster C_i using a weighted average:

$$v' = \frac{\text{sum}(\{v \times s \mid (v, s) \in C_i\})}{\text{sum}(\{s \mid (_, s) \in C_i\})}$$

Intuitively, this method samples v' around higher-scoring values within C_i , with the sampling range spanning from the smallest value to the largest value in C_i . For example, if the first cluster

$\{(2GB, 28), (3GB, 26)\}$ from Equation (2) is selected for the max-memory parameter, which sets the upper limit of memory usage, the newly sampled value is computed as follows:

$$v'_{\text{mem}} = \frac{(2GB \times 28) + (3GB \times 26)}{28 + 26} \approx 2.48GB$$

In this way, the Exploit strategy systematically samples a new value v' from SP_p and adds (p, v') to the set *PV* of parameter values at line 9. This process is repeated for parameters assigned to the Exploit strategy, which focuses on exploiting the accumulated data.

After sampling values *PV* for all selected parameters using either the Explore or Exploit strategy, Algorithm 1 executes symbolic execution on the target program with *PV*. It computes the set *B* of covered branches (line 21), adds a singleton set containing *B* to *TotalB* (line 22), and accumulates each triplet (p, v, B) —corresponding to each parameter p and its sampled value v in *PV* to *D* (line 23). Algorithm 1 continues the loop from lines 13–24 until the total budget expires. Upon completion, it returns the cumulative set *UnionB* of covered branches based on the sets accumulated in *TotalB* (i.e., $\text{UnionB} = \bigcup_{B \in \text{TotalB}}$). In this way, PARASUIT selects appropriate parameters and assigns their plausible values tailored to the target program based on the accumulated data while interacting with the symbolic execution tool. Ultimately, PARASUIT enables more effective testing of each program without any user-driven configuration.

4 Experiments

We evaluate PARASUIT to answer the following research questions:

- **Branch coverage:** To what extent does PARASUIT improve branch coverage over manual and user-configured tuning methods?
- **Bug-finding ability:** How effectively can PARASUIT find unique bugs compared to existing tuning approaches?
- **Efficacy of two stages:** Does our parameter selection algorithm outperform naive approaches? Is our value sampling stage more effective than generic sampling methods?
- **Analysis of self-configuration:** Which parameters are most impactful? How do they and their values vary by program? Does the new setting sampled by PARASUIT surpass existing baselines?
- **Generality:** Can PARASUIT be applied to concolic testing tools?

4.1 Experimental Settings

Baselines. We compared PARASUIT with four baselines: KLEE_{fix}, SYMTUNER, SYMTUNER_{AllP}, and RANDSUIT. First, KLEE_{fix} uses a fixed parameter setting as shown in Figure 1-(a), commonly adopted in prior works [22, 23, 35, 87]. For a clear comparison, we ran KLEE_{fix} with two modes: once for the total budget (24 hours) without early termination and multiple times with a short budget (120 seconds), similar to PARASUIT (Algorithm 1). We then reported the better result. The second baseline, SYMTUNER [20], is the state-of-the-art approach for parameter tuning in symbolic execution. It requires 20 user-configured parameter spaces, as depicted in Figure 1-(b). SYMTUNER_{AllP}, a variant of SYMTUNER, uses all 90 parameters extracted by the Initialize function, in line 4 of Algorithm 1. Similar to what SYMTUNER defined in Figure 1-(b), we defined the parameter space by selecting 5 to 10 values for each of the remaining 70 parameters, chosen to be either higher or lower relative to the default value. The last baseline, RANDSUIT, is a naive version of PARASUIT (Algorithm 1) that randomly selects parameters and samples their

Table 1: 12 benchmarks used in the experiments.

Program	LoC	# Branches	Program	LoC	# Branches
xorriso-1.5.2	161K	49,162	enscript-1.6.6	49K	3,796
gcal-4.1	89K	15,799	combine-0.4.0	32K	2,357
grep-3.4	82K	8,225	trueprint-5.4	12K	2,518
gawk-5.1.0	77K	10,720	diff-3.7	9K	7,612
sed-4.8	66K	6,798	du-8.32	8K	6,653
nano-4.9	54K	10,436	ls-8.32	5K	3,776

respective values. Precisely, RANDSUIT randomly selects parameters to tune from the 90 available parameters without executing the Extraction stage (lines 5–11 in Algorithm 1). It then assigns their values using only the Explore strategy (line 7 of Algorithm 2), skipping the Exploit strategy, including the clustering process.

Other Settings. We tested the performance of PARASUIT using 12 GNU open-source C programs previously used in SYMTUNER [20]. Given that SYMTUNER is an arguably strong baseline, this choice ensures a rigorous evaluation for PARASUIT. Table 1 presents the lines of code (LoC) and total branch count for each benchmark. Note that these benchmarks have also been widely used in existing works [12, 13, 18, 44, 54, 80] for improving the effectiveness of symbolic execution. The total testing budget was set at 24 hours. All experiments were replicated five times, and we reported the average results. We have built PARASUIT on top of KLEE-2.1 [14], a popular symbolic execution tool. We ran every experiment on a Linux machine equipped with two Intel Xeon Gold 6230 processors.

4.2 Branch Coverage

Table 2 demonstrates that PARASUIT consistently outperforms both static and user-defined configuration approaches in terms of branch coverage across all 12 benchmarks. We used gcov to collect the covered branches for each benchmark. In summary, PARASUIT covered 25.8% more branches than SYMTUNER, 97.4% more than KLEE_{fix}, 46.8% more than RANDSUIT, and 95.6% more than SYMTUNER_{AllP}.

Notably, PARASUIT, our self-configuring approach, consistently achieves higher branch coverage than SYMTUNER, which relies on manually defined configurations. For instance, in the case of the largest program `xorriso`, PARASUIT achieved 106.6% higher branch coverage than SYMTUNER, the second-best baseline—making the largest difference across all benchmarks. Interestingly, in `grep`, SYMTUNER even covered 2% fewer branches than RANDSUIT, which naively selects parameters and their corresponding values. This finding highlights a key limitation of SYMTUNER—its use of a pre-defined configuration regardless of the target program.

The results of SYMTUNER and SYMTUNER_{AllP} underscore the complexity of manual configuration. Excluding PARASUIT, SYMTUNER consistently achieved the highest branch coverage on 11 of the 12 benchmarks. However, SYMTUNER_{AllP} ranked the lowest in branch coverage on 8 out of the 12 benchmarks. The only difference between SYMTUNER and SYMTUNER_{AllP} lies in the number of parameter spaces utilized: 20 for SYMTUNER and 90 for SYMTUNER_{AllP}. These results highlight the user challenges of selecting 20 relevant parameters out of a possible 90 and then defining suitable candidate values for those selected parameters.

The coverage results of PARASUIT are statistically significant. This is confirmed by the Mann-Whitney U test [63], with p-values

Table 2: Branch coverage achieved by PARASUIT and four baselines on 12 benchmarks.

Benchmark	PARASUIT	SYMTUNER	KLEE _{fix}	RANDSUIT	SYMTUNER _{AllP}
xorriso-1.5.2	6,109.4	2,956.8	2,014.8	2,272.8	2,117.6
gcal-4.1	4,766.0	3,907.0	2,003.2	3,234.6	1,825.0
grep-3.4	3,613.8	3,280.2	2,208.4	3,344.2	2,017.8
gawk-5.1.0	4,393.2	3,747.0	2,841.6	2,893.0	3,261.8
sed-4.8	2,462.6	2,357.8	1,097.0	1,840.8	1,267.6
nano-4.9	1,909.6	1,721.0	1,457.0	1,662.2	1,289.0
enscript-1.6.6	776.4	734.2	580.8	688.8	448.6
combine-0.4.0	1,004.6	952.4	644.8	835.6	463.4
trueprint-5.4	1,300.2	1,019.2	579.0	881.6	571.2
diff-3.7	2,139.0	1,762.2	538.4	1,462.4	1,034.2
du-8.32	1,238.6	1,190.6	829.2	1,187.8	814.0
ls-8.32	2,165.4	1,712.0	1,355.2	1,410.4	1,185.8
total	31,878.8	25,340.4	16,149.4	21,714.2	16,296.0

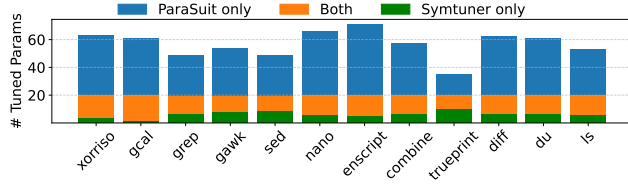
below 0.05 in all 48 cases (12 benchmarks $\times$ 4 baselines). The standard deviations of branch coverage across five repetitions of 12 benchmarks were as: PARASUIT (57), SYMTUNER (97), KLEE_{fix} (41), RANDSUIT (109), and SYMTUNER_{AllP} (250). This indicates that our results were relatively stable.

PARASUIT can also be augmented with prior knowledge, which further enhances its effectiveness. Across 12 programs, we conducted an experiment where we (1) removed approximately 34 unimpactful parameters from the 90 candidates (e.g., `|P|` at line 4 in Algorithm 1), and (2) replaced default initial values with more efficient ones, using knowledge from PARASUIT’s earlier 24-hour run. These adjustments led to 5.7% higher average branch coverage. These results suggest that, when provided with high-quality prior knowledge, PARASUIT’s effectiveness can be further improved.

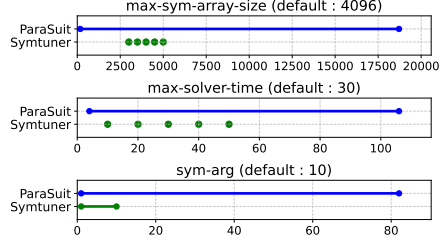
Comparison with User-configured Approach. To demonstrate how PARASUIT achieved higher branch coverage than the state-of-the-art approach (SYMTUNER), we analyzed the differences in selected parameters and values. Figure 3-(a) shows that PARASUIT consistently selects more diverse parameter sets per program compared to SYMTUNER. On average, PARASUIT tunes 50.3 parameters per program, 72.9% of which were not selected by SYMTUNER. For example, in `trueprint`, it covered 28.7% more branches by excluding 10 parameters selected by SYMTUNER and uniquely tuning 15 others. These results indicate that PARASUIT effectively tailors parameter selection to each program’s unique characteristics.

Figure 3-(b) illustrates PARASUIT samples a wider range of values compared to SYMTUNER. For the three integer parameters frequently selected across the programs, PARASUIT samples values from a range that is, on average, 6.9 times wider than SYMTUNER. Furthermore, PARASUIT samples efficient values from a range-based search space, whereas SYMTUNER uses a fixed set of five values for each parameter. Interestingly, the best-performing values for these parameters in PARASUIT were outside the space defined by SYMTUNER. For example, the most promising values for `sym-arg` discovered by PARASUIT—such as 41 for `xorriso`, 21 for `gcal`, and 37 for `grep`—were ones that SYMTUNER failed to assign.

Comparison with 24-Hour Continuous Run. To examine whether PARASUIT’s iterative approach—running symbolic execution multiple times within shorter time budgets—limits its ability to explore deeper paths, we compared it with KLEE_{24h}, a baseline



(a) The difference in the parameter selection.



(b) The difference in the value sampling.

Figure 3: The differences in parameter tuning between PARASUIT and user-configured approach.

that runs KLEE continuously for 24 hours. We evaluated each baseline based on (1) the depth of test-cases, indicating how deeply each method can reach the execution paths, and (2) the number of uniquely covered branches, which reflects the range of exploration.

In our evaluation, PARASUIT consistently outperforms KLEE_{24h} in both criteria. In terms of test-case depth, PARASUIT explored symbolic branches that were, on average, 123.9% deeper, and reached up to 8.1 times deeper in the maximum depth comparison. Furthermore, PARASUIT missed only 7.5 branches on average that were uniquely covered by KLEE_{24h}, while discovering 1689.4 branches that KLEE_{24h} failed to reach. Notably, in 9 benchmarks, the set of branches covered by PARASUIT was a strict superset of those covered by KLEE_{24h}. These improvements are largely attributed to the use of effective parameters such as "seed-file". This parameter enables PARASUIT to utilize high-scoring inputs—ranked using our customized scoring function—as seed inputs for subsequent symbolic execution runs. As a result, PARASUIT can bypass computationally expensive SMT solving steps, allowing it to reach progressively deeper regions of the code more efficiently.

4.3 Bug-Finding Ability

Table 3 shows the effectiveness of PARASUIT in identifying unique bugs compared to other baselines. To obtain these results, we collected all test-cases flagged by KLEE as bugs, denoted as the "*.err" files. We then re-executed the original binary of each program with these potential bug-triggering test-cases and retained only those that were reproducible. A bug-triggering test-case was considered unique if its error location reported by KLEE differed from others. In the experiment, we reported identified bugs within the total testing budget (24 hours $\times$ 5 repetitions).

In total, PARASUIT succeeded in detecting 11 unique bugs across three of the 12 programs: gcal, gawk, and combine. This count is 4 more than the state-of-the-art SYMTUNER; PARASUIT surpassed the three baselines by finding 6, 7, and 8 more unique bugs than KLEE_{fix},

Table 3: Number of bugs detected by PARASUIT and baselines.

Benchmark	PARASUIT	SYMTUNER	KLEE _{fix}	RANDSUIT	SYMTUNER _{ALLP}
gcal	7	4	4	2	1
gawk	1	1	-	1	-
combine	3	2	1	1	1
total	11	7	5	4	3

RANDSUIT, and SYMTUNER_{ALLP}, respectively. Notably, PARASUIT did not miss any bugs that other baselines detected, while it successfully identified four additional bugs. All bugs discovered by PARASUIT were reported to the program developers, and the bug found in gawk was confirmed.

4.4 Efficacy of Two Stages

Parameter Selection Stage. We evaluated the effectiveness of our parameter selection stage (lines 15–17 in Algorithm 1), which is designed to automatically select promising parameters suited to the program under test. We compared its performance in terms of bug detection and branch coverage with two simpler variants of PARASUIT that replace our parameter selection stage, while keeping all other components unchanged: RANDP and ALLP. RANDP randomly selects parameters to be tuned from all available parameters extracted by the Initialize function (line 4 in Algorithm 1). In contrast, ALLP selects all available parameters, proceeding directly to the value sampling stage without any filtering. In addition, we compared our algorithm against SYMP, which selects the same 20 parameters manually chosen by SYMTUNER, as shown in Figure 1–(b). We conducted these comparisons on five benchmarks, covering the three largest programs (xorriso, gcal, grep) and the three programs where bugs were identified (gcal, gawk, combine).

Table 4 demonstrates that our parameter selection algorithm surpasses the three simpler selection approaches. Across five programs, PARASUIT covered 60.2%, 89.9%, and 34.2% more branches on average than RANDP, ALLP, and SYMP, respectively. The bug-finding efficiency was even more pronounced; RANDP identified only 3 bugs, whereas ALLP and SYMP detected 7 and 5 fewer than PARASUIT, respectively. In particular, the comparison between PARASUIT and SYMP highlights the importance of selecting parameters in a program-specific manner. The clear improvement over SYMP indicates that PARASUIT successfully selects the most effective parameters for each program without imposing any burden on the user.

Value Sampling Stage. We also assessed the efficiency of our value sampling stage that iteratively constructs sampling spaces for selected parameters and updates their sampling probabilities. This was compared to four sampling algorithms commonly used in search-based software engineering [24, 28, 43, 67]. Specifically, after completing the parameter selection stage of PARASUIT, we substituted our value sampling stage (the SAMPLE procedure at line 20 of Algorithm 1) with three widely-used algorithms: BAYESIAN (Bayesian optimization) [88], GENETIC (Genetic algorithm) [46], and CMA-ES (Covariance Matrix Adaptation-Evolution Strategy) [41]. To ensure fair comparisons, we set each parameter's sampling range to the interval $[lv, hv]$, spanning the smallest to largest values sampled by PARASUIT within the total 24-hour budget. GENETIC used a real-coded genetic algorithm with simulated binary crossover [31],

Table 4: Effectiveness of our parameter selection stage compared with naive selection approaches.

Benchmark	PARASUIT	RANDP	SYMP	ALLP
xorriso	6,109(0)	2,615(0)	3,149(0)	2,426(0)
gcal	4,766(7)	3,578(2)	4,160(3)	3,105(2)
grep	3,614(0)	2,852(0)	3,404(0)	1,749(0)
gawk	4,393(1)	2,551(0)	3,336(1)	2,521(1)
combine	1,005(3)	861(1)	953(2)	577(1)
total	19,887(11)	12,457(3)	15,002(6)	10,378(4)

Table 5: The efficiency of the value sampling algorithm compared with five sampling methods.

Benchmark	PARASUIT	BAYESIAN	GENETIC	CMA-ES	RANDV	SYMV
xorriso	6,109(0)	4,417(0)	4,383(0)	5,361(0)	3,759(0)	2,745(0)
gcal	4,766(7)	4,617(5)	4,552(5)	4,412(5)	4,421(4)	3,642(2)
grep	3,614(0)	3,559(0)	3,349(0)	3,455(0)	3,407(0)	3,414(0)
gawk	4,393(1)	4,116(1)	3,902(0)	3,801(1)	3,920(0)	2,694(0)
combine	1,005(3)	883(1)	922(1)	892(2)	868(1)	856(1)
total	19,887(11)	17,592(7)	17,108(6)	17,921(8)	16,375(5)	13,351(3)

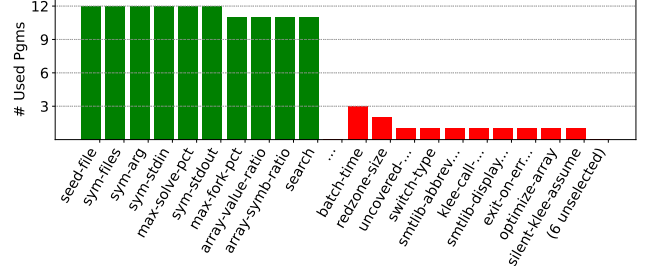
while the others followed their standard approaches. We additionally included RANDV, which samples values with only the Explore strategy (line 7 in Algorithm 2), and SYMV, which applies the sampling algorithm of SYMTUNER [20] to the parameters selected by PARASUIT. The experiments used the same five benchmarks as those used in the evaluation of our parameter selection stage.

Table 5 shows that our customized sampling algorithm outperforms all six parameter sampling methods. Across five benchmarks, PARASUIT achieved, on average, 11.0% higher branch coverage and identified three additional bugs than the second-best method (CMA-ES). PARASUIT also achieved 21.5% higher branch coverage than RANDV, while detecting six more bugs. The results from RANDV demonstrate that it is essential not only to flexibly define sampling spaces but also to employ intelligent strategies—such as our Exploit strategy (line 9 in Algorithm 2)—for sampling values within those spaces. PARASUIT also achieved 49.0% higher branch coverage and detected eight more unique bugs than SYMV. This result is particularly noteworthy, as it was achieved even when SYMV, a variant of SYMTUNER, was allowed to dynamically select promising parameters using our parameter selection stage. Nevertheless, PARASUIT still outperforms SYMV, which uses its own value sampling algorithm specialized for tuning symbolic execution parameters. The success of PARASUIT is attributed to its program-aware sampling spaces and value sampling strategies, unlike SYMV, which relies on user-defined sampling spaces.

To further validate the effectiveness of PARASUIT’s value sampling stage, we compared it against two state-of-the-art configurators: ParamILS [50] and OpenTuner [2]. We modified these configurators to sample values in each iteration using their respective algorithms. Across the same five benchmarks presented in Table 5, PARASUIT achieved 14.2% higher branch coverage on average and uncovered five additional bugs compared to OpenTuner, the second-best configurator. This limitation arises because such tools typically rely on offline learning over predefined parameter spaces to find a single effective configuration, and they require substantial prior data to operate effectively.

Table 6: Top-5 parameters with the greatest impact on symbolic execution across five benchmarks.

	xorriso	gcal	grep	gawk	combine
1	seed-dir	use-indep.-solver	sym-arg	seed-file	max-cpfork-pct
2	max-fork-pct	sym-arg	max-solve-pct	sym-files	array-symb-ratio
3	array-value-ratio	seed-file	max-cpsolve-pct	search	max-solver-time
4	seed-file	max-cpfork-pct	sym-files	sym-stdin	sym-arg
5	array-symb-ratio	max-solve-pct	max-fork-pct	sym-arg	sym-stdin

**Figure 4: The count of benchmarks where each parameter was chosen at least once.**

4.5 Analysis of Self-configuration

Impact of Selected Parameters. We explored the impact of individual parameters selected by PARASUIT on its performance across five representative programs: xorriso, gcal, grep, gawk, and combine. For each program, we measured the first-order Sobol indices [79] using the accumulated data—specifically, D at line 23 in Algorithm 1—collected over 120 hours (i.e., 24 hours $\times$ 5 trials). These indices quantify how much each parameter contributes to the variance in branch coverage during the experiments, reflecting both its individual and synergetic effects with other parameters. Each index ranges between 0 and 1, with higher values indicating greater importance in accurately tuning the corresponding parameter.

Table 6 reveals that the most impactful parameters vary by program. It lists the top 5 index-ranked parameters for each of the five benchmarks. Interestingly, no parameter consistently ranked among the top five across all five benchmarks. The most recurring was `sym-arg`—which determines the number and length of symbolic arguments—ranked in four benchmarks. Notably, although the most impactful parameter for `gcal` was `use-independent-solver`, this parameter did not rank in top-5 for other programs: `xorriso` (53), `grep` (27), `gawk` (37) and `combine` (32). These results demonstrate that PARASUIT’s effectiveness stems from its ability to reduce the 90-dimensional parameter space in a program-specific manner and focus tuning on the most impactful parameters.

Tendency of Selections and Sampled Values. We analyzed how much the selected parameters and their values, as tuned by PARASUIT, vary depending on the target program. Figure 4 shows that although a general pattern in parameter selection and non-selection is observed across our 12 benchmarks, selecting the right parameters for each specific benchmark is crucial for optimal symbolic execution. Notably, six parameters were consistently selected across all benchmarks due to their efficiency, and another six parameters were never selected in any of the benchmarks. Together, these 12 parameters account for only 13.3% of the total 90 parameters.

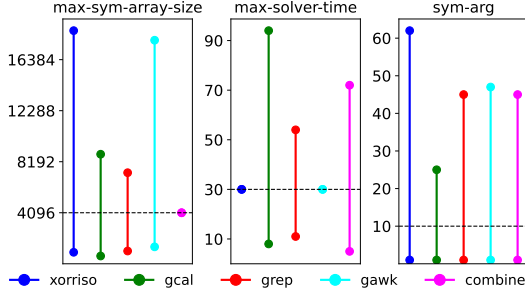


Figure 5: Integer parameter values sampled by PARASUIT.

For the remaining 77 parameters, the selection varied significantly between benchmarks. This suggests that the effectiveness of a particular parameter can be benchmark-specific, emphasizing the need for parameter selection customized to each program.

Figure 5 illustrates that the values sampled by PARASUIT vary significantly across benchmarks; for parameters not selected in a given benchmark (e.g., max-solver-time in xorriso), their range is represented as a single point corresponding to the parameter's default value. We compared the range of sampled values for three integer parameters which were selected frequently by 12 benchmarks in Table 1. The results show that the range of parameter values differs depending on each benchmark. For instance, in xorriso, PARASUIT samples the value of "max-sym-array-size" from a range of 17,791, which is approximately 3.3 times larger than the range for combine. Similarly, PARASUIT does not select the max-solver-time parameter for tuning on the xorriso and gawk programs, while sampling its value from a range of 86 in gcal. The parameters with the largest range of values also varied depending on the benchmarks. For example, "max-solver-time" had the widest range in gcal, while "sym-arg" exhibited the widest ranges for xorriso.

Efficiency of Self-configured Parameter Sets. We investigated whether the results are effective by defining $KLEE_{new}$ with PARASUIT's configuration and comparing it to $KLEE_{fix}$, which uses the fixed parameters in Figure 1-(a). To construct this new program-specific parameter setting, we first selected 31 parameters—equal in number to those in $KLEE_{fix}$ —based on Sobol indices. We then assigned the highest-scoring value for each selected parameter. Finally, for each program, we ran $KLEE_{new}$ in two modes and reported the better result, following the same setup used for $KLEE_{fix}$.

Our results demonstrate that utilizing the newly defined parameter settings consistently outperforms the traditional $KLEE_{fix}$ configuration. $KLEE_{new}$ achieved, on average, 16.5% higher branch coverage than $KLEE_{fix}$ and uncovered one additional bug in gawk. This demonstrates that PARASUIT is capable of automatically identifying superior parameter settings for the target programs from scratch. Despite the effectiveness of $KLEE_{new}$, it still covered 43.1% fewer branches and missed seven bugs compared to PARASUIT, which iteratively explores a variety of parameter sets for short durations. This indicates that trying multiple parameter sets over short periods is more efficient than running symbolic execution with a single fixed parameter setting, as each configuration can reach different regions of the target program's code.

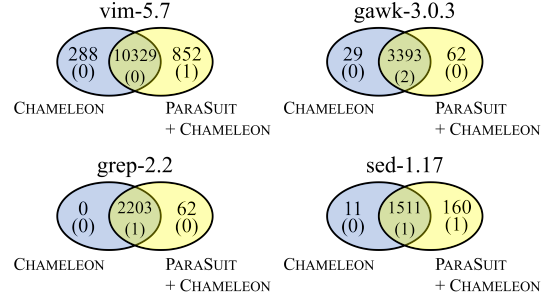


Figure 6: Efficacy of parameter tuning for concolic testing.

4.6 Generality

We investigated the potential for generalizing PARASUIT to another symbolic execution method, concolic testing, which has been widely used in prior works [10, 21, 75]. Specifically, we integrated PARASUIT with CHAMELEON [21], a cutting-edge technique, to evaluate whether PARASUIT could further optimize their performance. PARASUIT aims to autonomously tune the entire set of five parameters in CHAMELEON. Given that CHAMELEON is designed to automatically adjust the "search" parameter during symbolic execution, our version, PARASUIT + CHAMELEON, permits CHAMELEON to tune this parameter using its own algorithm while concurrently tuning all other parameters. We used the same four largest benchmarks as CHAMELEON and set the testing budget to 24 hours. We reported the average number of covered branches and unique bugs identified by each technique over five trials.

Figure 6 shows that our approach significantly enhances the performance of concolic testing in identifying unique branches and bugs. Across all four benchmarks, PARASUIT + CHAMELEON covered 3.5 times more unique branches than CHAMELEON alone, on average. In particular, for grep, PARASUIT + CHAMELEON covered all branches reached by the untuned one and additionally reached more, forming a clear superset. In bug detection, PARASUIT + CHAMELEON discovered six distinct bugs, including two additional bugs in vim and sed. These results—consistent across tools like KLEE, which has over 100 parameters, and CHAMELEON, with only five—demonstrate the capability of PARASUIT to significantly improve the performance of symbolic execution tools, irrespective of their parameter count.

We also found that integrating PARASUIT with other symbolic execution tools: angr [77] and SPF [68], requires only minor modifications. As with KLEE and CHAMELEON, PARASUIT can extract parameter names from the help scripts and default values from the abstract syntax tree of their source code. For the self-configuring stages—parameter selection and value sampling—PARASUIT requires only two pieces of information from the SE tool: the parameter values used and the corresponding test-cases generated in each iteration. Since both angr and SPF provide this data, we believe that integration with these tools is straightforward.

4.7 Threats to Validity

(1) PARASUIT uses two hyper-parameters— η_{time} (120 seconds) and η_{clust} (0.7)—as initial settings. To reduce sensitivity to these values, we tune them using the same program-specific method (Algorithm 2) applied to other external symbolic execution parameters. While more effective values may exist for individual programs, our evaluation shows found that PARASUIT’s performance is generally robust to such variations (e.g., only a 1% drop when η_{clust} is set to 0.9). (2) We evaluated the effectiveness of PARASUIT with 12 commonly-used C programs as benchmarks. Although these benchmarks cover a broad spectrum of code sizes and branch counts, they may not be representative of all types of programs.

5 Related Work

Enhancing Symbolic Execution. Our work belongs to a line of research that introduces unique techniques to improve the effectiveness of symbolic execution [6, 9, 10, 19, 44, 51, 57, 75, 84, 89, 91]. PARASUIT automates parameter configuration for symbolic execution, removing manual intervention. Unlike ours, previous works on search heuristics [10, 19, 44, 57, 75, 84] have aimed to enhance symbolic execution by prioritizing which states to explore from a vast pool of candidates. Each heuristic has its own criteria for selecting specific program states. For instance, the Subpath-Guided Search heuristic [57] is to prioritize states with fewer explored subpaths. Space reduction heuristics [6, 9, 51, 89, 91] aim to address the state-explosion problem by minimizing redundant program paths or states. Each heuristic introduces an approach to efficiently identify non-essential program paths. For example, post-conditioned symbolic execution [89] terminates redundant states based on already explored path suffixes. Beyond these, various methods like state-merging [27, 32, 56] and constraint solving reduction [32, 54, 69] have been proposed to optimize symbolic execution.

Tuning External Parameters. Our approach is rooted in automatic parameter tuning methods [2, 4, 5, 20, 47–50, 58, 59, 62, 64, 83]. Automatically tuning external parameters is growing in importance as tools in various fields evolve and become more sophisticated. As these tools incorporate more parameters to optimize performance, users often find it challenging to configure them. Consequently, numerous automatic tuning techniques, informed by domain knowledge, have emerged in fields like robotics [5, 59], machine learning models [61, 65, 86], big data processing systems [34, 45], static analysis [85], and symbolic execution [20]. However, unlike PARASUIT, which configures parameters using an online learning algorithm, existing approaches for tuning configurable systems rely on offline strategies that require substantial prior information for learning. For example, irace [58] typically requires around 5,000 evaluations—equivalent to 167 hours of symbolic execution—to converge, which makes it unsuitable for our setting. Both our approach, PARASUIT, and SYMTUNER [20] aim to adjust the external parameters of symbolic execution tools. The key difference is that PARASUIT automatically identifies and configures parameters in a program-specific manner, whereas SYMTUNER relies on manually defined spaces that are fixed and do not vary across programs.

Search-based Software Testing. Our approach falls within the scope of search-based software testing (SBST) [1, 17, 26, 36,

42, 52, 53, 70, 76, 92]. At a high-level, each specific testing task in SBST—such as improving coverage [36], usability [1], or fault detection [70]—is formulated as an optimization problem and addressed using a domain-specific algorithm tailored to the task. However, within the SBST framework, PARASUIT and prior techniques differ substantially in the goals they target. PARASUIT aims to maximize exploration by applying multiple efficient parameter values to the target program, whereas prior approaches [17, 52, 53] focus on quickly converging to a single efficient setting within a predefined space and generally remain fixed thereafter without further adjustment. In this context, we formulate an optimization problem that identifies sets of parameters and values ($PV_1, PV_2, \dots, PV_n$) such that the union of symbolic execution outcomes (e.g., branch coverage) is maximized within a fixed testing budget. We propose a specialized algorithm, tailored for symbolic execution, that dynamically selects promising parameters, constructs program-specific value sampling spaces for those selected, and defines two distinct sampling strategies based on data accumulated during interaction with the symbolic execution tool.

6 Conclusion

Modern symbolic execution tools incorporate a variety of parameters to boost their performance. Paradoxically, as the number of parameters grows, configuring them becomes increasingly complex for users. This paper presents a specialized algorithm designed to fully automate the program-aware parameter tuning process for symbolic execution. This algorithm autonomously selects impactful parameters, constructs their sampling spaces, and updates sampling probabilities, while iteratively interacting with the symbolic execution tool. The experimental results demonstrate that our approach (PARASUIT) outperforms both manual and user-configured approaches in terms of code coverage and the discovery of unique bugs, while eliminating the need for user-driven tuning. We expect our approach to offer substantial benefits to both end users and researchers by simultaneously enhancing the performance and usability of practical symbolic execution tools.

Acknowledgments

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. RS-2021-NR061858, No. RS-2023-00208094). This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2024-00438686, Development of software reliability improvement technology through identification of abnormal open sources and automatic application of DevSecOps). This work was partly supported by the Institute of Information & communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (RS-2019-II190421, AI Graduate School Support Program(Sungkyunkwan University)).

References

- [1] Anne Adams and Martina Angela Sasse. 1999. Users are not the enemy. *Commun. ACM* 42, 12 (1999), 40–46.
- [2] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O’Reilly, and Saman Amarasinghe. 2014. Opentuner: An extensible framework for program autotuning. In *Proceedings of the 23rd international conference on Parallel architectures and compilation*. 303–316.

- [3] Thanassis Avgerinos, Alexandre Rebert, Sang Kil Cha, and David Brumley. 2014. Enhancing Symbolic Execution with Veritesting. In *Proceedings of the 36th International Conference on Software Engineering (ICSE '14)*. 1083–1094.
- [4] Alessio Bechini, Martina Criscone, Pietro Ducange, Francesco Marcelloni, and Alessandro Renda. 2020. FDBSCAN-APT: a fuzzy density-based clustering algorithm with automatic parameter tuning. In *2020 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE)*. IEEE, 1–8.
- [5] Felix Berkenkamp, Andreas Krause, and Angela P Schoellig. 2021. Bayesian optimization with safety constraints: safe and automatic parameter tuning in robotics. *Machine Learning* (2021), 1–35.
- [6] Peter Boonstoppel, Cristian Cadar, and Dawson Engler. 2008. RWset: Attacking path explosion in constraint-based test generation. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS '08)*. 351–366.
- [7] Max Brunsfeld. 2025. Tree-sitter: A new parsing system for programming tools. <https://tree-sitter.github.io/tree-sitter/>
- [8] Suhabe Bugrara and Dawson Engler. 2013. Redundant state detection for dynamic symbolic execution. In *2013 USENIX Annual Technical Conference (USENIX ATC '13)*. 199–211.
- [9] Suhabe Bugrara and Dawson Engler. 2013. Redundant State Detection for Dynamic Symbolic Execution. In *Proceedings of the 2013 USENIX Conference on Annual Technical Conference (USENIX ATC '13)*. 199–212.
- [10] Jacob Burnim and Koushik Sen. 2008. Heuristics for Scalable Dynamic Test Generation. In *Proceedings of 23rd IEEE/ACM International Conference on Automated Software Engineering (ASE '08)*. 443–446.
- [11] Frank Busse, Pritam Gharat, Cristian Cadar, and Alastair F Donaldson. 2022. Combining static analysis error traces with dynamic symbolic execution (experience paper). In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 568–579.
- [12] Frank Busse, Pritam Gharat, Cristian Cadar, and Alastair F. Donaldson. 2022. Combining Static Analysis Error Traces with Dynamic Symbolic Execution (Experience Paper). In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2022)*. 568–579.
- [13] Frank Busse, Martin Nowack, and Cristian Cadar. 2020. Running symbolic execution forever. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 63–74.
- [14] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI '08)*. 209–224.
- [15] Cristian Cadar and Dawson Engler. 2005. Execution Generated Test Cases: How to Make Systems Code Crash Itself. In *Proceedings of the 12th International Conference on Model Checking Software (SPIN'05)*. 2–23.
- [16] Cristian Cadar, Vijay Ganesh, Peter M. Pawlowski, David L. Dill, and Dawson R. Engler. 2008. EXE: Automatically Generating Inputs of Death. *ACM Trans. Inf. Syst. Secur.* 12, 2 (2008), 10:1–10:38.
- [17] Rong Cao, Liang Bao, Kaibi Zhao, and Panpan Zhangsun. 2024. ETune: Efficient configuration tuning for big-data software systems via configuration space reduction. *Journal of Systems and Software* 209 (2024), 111936.
- [18] Sooyoung Cha, Seongjoon Hong, Jiseong Bak, Jingyoung Kim, Junhee Lee, and Hakjoo Oh. 2022. Enhancing Dynamic Symbolic Execution by Automatically Learning Search Heuristics. *IEEE Transactions on Software Engineering* (2022), 3640–3663.
- [19] Sooyoung Cha, Seongjoon Hong, Junhee Lee, and Hakjoo Oh. 2018. Automatically Generating Search Heuristics for Concolic Testing. In *Proceedings of the 40th International Conference on Software Engineering (ICSE '18)*. 1244–1254.
- [20] Sooyoung Cha, Myungho Lee, Seokhyun Lee, and Hakjoo Oh. 2022. SymTuner: Maximizing the Power of Symbolic Execution by Adaptively Tuning External Parameters. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. 2068–2079.
- [21] Sooyoung Cha and Hakjoo Oh. 2019. Concolic Testing with Adaptively Changing Search Heuristics. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '19)*. 235–245.
- [22] Sooyoung Cha and Hakjoo Oh. 2020. Making Symbolic Execution Promising by Learning Aggressive State-Pruning Strategy. In *The 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*.
- [23] Junjie Chen, Wenxiang Hu, Lingming Zhang, Dan Hao, Sarfraz Khurshid, and Lu Zhang. 2018. Learning to accelerate symbolic execution via code transformation. In *32nd European Conference on Object-Oriented Programming (ECOOP '18)*.
- [24] Junjie Chen, Ningxin Xu, Peiqi Chen, and Hongyu Zhang. 2021. Efficient compiler autotuning via bayesian optimization. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1198–1209 (2021).
- [25] Yizong Cheng. 1995. Mean shift, mode seeking, and clustering. *IEEE transactions on pattern analysis and machine intelligence* (1995), 790–799.
- [26] Ankur Choudhary, Arun Prakash Agrawal, and Arvinder Kaur. 2018. An effective approach for regression test case selection using pareto based multi-objective harmony search. In *Proceedings of the 11th International Workshop on Search-Based Software Testing*. 13–20.
- [27] Philippe Codognet and Daniel Diaz. 2001. Yet another local search method for constraint solving. In *International Symposium on Stochastic Algorithms*. 73–90.
- [28] Thelma Elita Colanzi, Wesley KG Assunção, Silvia R Vergilio, Paulo Roberto Farah, and Giovanni Guizzo. 2020. The symposium on search-based software engineering: Past, present and future. *Information and Software Technology* 127 (2020), 106372.
- [29] CREST. A concolic test generation tool for C. 2008. <https://github.com/jburnim/crest>.
- [30] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS '08)*. 337–340.
- [31] Kalyanmoy Deb and Hans-Georg Beyer. 2001. Self-adaptive genetic algorithms with simulated binary crossover. *Evolutionary computation* 9, 2 (2001), 197–221.
- [32] Peter Dinges and Gul Agha. 2014. Solving Complex Path Conditions Through Heuristic Search on Induced Polytopes. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE '14)*. 425–436.
- [33] Zhen Dong, Artur Andrzejak, David Lo, and Diego Costa. 2016. Orplocator: Identifying read points of configuration options via static analysis. In *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 185–195.
- [34] Hui Dou, Lei Zhang, Yiwen Zhang, Pengfei Chen, and Zibin Zheng. 2023. TurBO: A cost-efficient configuration-based auto-tuning approach for cluster-based big data frameworks. *J. Parallel and Distrib. Comput.* 177 (2023), 89–105.
- [35] Oscar Soria Dustmann, Klaus Wehrle, and Cristian Cadar. 2018. PARTI: A Multi-Interval Theory Solver for Symbolic Execution. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE '18)*. 430–440.
- [36] Gordon Fraser and Andrea Arcuri. 2012. The seed is strong: Seeding strategies in search-based software testing. In *2012 IEEE fifth international conference on software testing, verification and validation*. IEEE, 121–130.
- [37] Vijay Ganesh and David L Dill. 2007. A decision procedure for bit-vectors and arrays. In *International Conference on Computer Aided Verification (CAV '07)*. 519–531.
- [38] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: Directed Automated Random Testing. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '05)*. 213–223.
- [39] Patrice Godefroid, Michael Y. Levin, and David Molnar. 2012. SAGE: Whitebox Fuzzing for Security Testing: SAGE Has Had a Remarkable Impact at Microsoft. *Queue* (2012), 20–27.
- [40] Patrice Godefroid, Michael Y Levin, and David A Molnar. 2008. Automated Whitebox Fuzz Testing. In *Proceedings of the Symposium on Network and Distributed System Security (NDSS '08)*. 151–166.
- [41] Nikolaus Hansen and Andreas Ostermeier. 2001. Completely derandomized self-adaptation in evolution strategies. *Evolutionary computation* 9, 2 (2001), 159–195.
- [42] Mark Harman, Yue Jia, and Yuanyuan Zhang. 2015. Achievements, open problems and challenges for search based software testing. In *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 1–12.
- [43] Mark Harman, S Afshin Mansouri, and Yuanyuan Zhang. 2012. Search-based software engineering: Trends, techniques and applications. *ACM Computing Surveys (CSUR)* 45, 1 (2012), 1–61.
- [44] Jingxuan He, Gishor Sivanrupan, Petar Tsankov, and Martin Vechev. 2021. Learning to Explore Paths for Symbolic Execution. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS '21)*. 2526–2540.
- [45] Herodotos Herodotou, Yuxing Chen, and Jiaheng Lu. 2020. A survey on automatic parameter tuning for big data processing systems. *ACM Computing Surveys (CSUR)* 53, 2 (2020), 1–37.
- [46] John H Holland. 1992. Genetic algorithms. *Scientific american* 267, 1 (1992), 66–73.
- [47] Changwu Huang, Bo Yuan, Yuanxiang Li, and Xin Yao. 2019. Automatic parameter tuning using bayesian optimization method. In *2019 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2090–2097.
- [48] Frank Hutter, Holger Hoos, and Kevin Leyton-Brown. 2014. An efficient approach for assessing hyperparameter importance. In *International conference on machine learning*. PMLR, 754–762.
- [49] Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. 2011. Sequential model-based optimization for general algorithm configuration. In *International conference on learning and intelligent optimization*. Springer, 507–523.
- [50] Frank Hutter, Holger H Hoos, Kevin Leyton-Brown, and Thomas Stützle. 2009. ParamILS: an automatic algorithm configuration framework. *Journal of artificial intelligence research* 36 (2009), 267–306.
- [51] Joxan Jaffar, Vijayaraghavan Murali, and Jorge A. Navas. 2013. Boosting Concolic Testing via Interpolation. In *Proceedings of the 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE '13)*. 48–58.

- [52] Pooyan Jamshidi, Miguel Velez, Christian Kästner, and Norbert Siegmund. 2018. Learning to sample: Exploiting similarities across environments to learn performance models for configurable systems. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 71–82.
- [53] Christian Kaltenecker, Stefan Mühlbauer, Alexander Grebhahn, Norbert Siegmund, and Sven Apel. 2023. Performance evolution of configurable software systems: an empirical study. *Empirical Software Engineering* 28, 6 (2023), 152.
- [54] Timotej Kapus, Frank Busse, and Cristian Cadar. 2020. Pending Constraints in Symbolic Execution for Better Exploration and Seeding. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20)*. 115–126.
- [55] Volodymyr Kuznetsov, Johannes Kinder, Stefan Bucur, and George Candea. 2012. Efficient State Merging in Symbolic Execution. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '12)*. 193–204.
- [56] Xin Li, Yongjuan Liang, Hong Qian, Yi-Qi Hu, Lei Bu, Yang Yu, Xin Chen, and Xuandong Li. 2016. Symbolic Execution of Complex Program Driven by Machine Learning Based Constraint Solving. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE '16)*. 554–559.
- [57] You Li, Zhendong Su, Linzhang Wang, and Xuandong Li. 2013. Steering Symbolic Execution to Less Traveled Paths. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages, and Applications (OOPSLA '13)*. 19–32.
- [58] Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Mauro Biratari, and Thomas Stützle. 2016. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives* 3 (2016), 43–58.
- [59] Antonio Loquercio, Alessandro Saviolo, and Davide Scaramuzza. 2022. Autotune: Controller tuning for high-speed flight. *IEEE Robotics and Automation Letters* 7, 2 (2022), 4432–4439.
- [60] Sicheng Luo, Hui Xu, Yanxiang Bi, Xin Wang, and Yangfan Zhou. 2021. Boosting symbolic execution via constraint solving time prediction (experience paper). In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 336–347.
- [61] Yuanfei Luo, Mengshuo Wang, Hao Zhou, Quanming Yao, Wei-Wei Tu, Yuqiang Chen, Wenyuan Dai, and Qiang Yang. 2019. Autocross: Automatic feature crossing for tabular data in real-world applications. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1936–1945.
- [62] Gaddam Malleshham and Akula Rajani. 2006. Automatic tuning of PID controller using fuzzy logic. In *8th International Conference on Development and Application Systems*. 120–127.
- [63] Henry B Mann and Donald R Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics* (1947), 50–60.
- [64] MM Molina, JM Luna, C Romero, and S Ventura. 2012. Meta-Learning Approach for Automatic Parameter Tuning: A Case Study with Educational Datasets. *International Educational Data Mining Society* (2012).
- [65] Kristin Nielsen, Caroline Svahn, Hector Rodriguez-Deniz, and Gustaf Hendebý. 2021. Ukf parameter tuning for local variation smoothing. In *2021 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*. IEEE, 1–8.
- [66] OSDI'08_Coreutil_Experiments. 2008. <https://klee.github.io/docs/coreutils-experiments>.
- [67] Annibale Panichella. 2021. A systematic comparison of search-based approaches for LDA hyperparameter tuning. *Information and Software Technology* 130 (2021), 106411.
- [68] Corina S Păsăreanu and Neha Rungta. 2010. Symbolic PathFinder: symbolic execution of Java bytecode. In *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering*. 179–180.
- [69] David M. Perry, Andrea Mattavelli, Xiangyu Zhang, and Cristian Cadar. 2017. Accelerating Array Constraints in Symbolic Execution. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '17)*. 68–78.
- [70] Justyna Petke, Shin Yoo, Myra B Cohen, and Mark Harman. 2013. Efficiency and early fault detection with lower and higher strength combinatorial interaction testing. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. 26–36.
- [71] Ariel Rabkin and Randy Katz. 2011. Static extraction of program configuration options. In *Proceedings of the 33rd International Conference on Software Engineering*. 131–140.
- [72] Peter J Rousseeuw. 1987. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics* 20 (1987), 53–65.
- [73] Koushik Sen, Darko Marinov, and Gul Agha. 2005. CUTE: A Concolic Unit Testing Engine for C. In *Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE '05)*. 263–272.
- [74] Koushik Sen, George Necula, Liang Gong, and Wontae Choi. 2015. MultiSE: Multi-path Symbolic Execution Using Value Summaries. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE '15)*. 842–853.
- [75] Hyunmin Seo and Sunghun Kim. 2014. How We Get There: A Context-guided Search Strategy in Concolic Testing. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE '14)*. 413–424.
- [76] August Shi, Alex Gyori, Milos Gligoric, Andrey Zaytsev, and Darko Marinov. 2014. Balancing trade-offs in test-suite reduction. In *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*. 246–256.
- [77] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Audrey Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. 2016. SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis. In *IEEE Symposium on Security and Privacy*.
- [78] Ziqi Shuai, Zhenbang Chen, Kelin Ma, Kunlin Liu, Yufeng Zhang, Jun Sun, and Ji Wang. 2024. Partial Solution Based Constraint Solving Cache in Symbolic Execution. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2493–2514.
- [79] Ilya M Sobol. 2001. Global sensitivity indices for nonlinear mathematical models and their Monte Carlo estimates. *Mathematics and computers in simulation* 55, 1–3 (2001), 271–280.
- [80] Yue Sun, Guowei Yang, Shichao Lv, Zhi Li, and Limin Sun. 2024. Concrete Constraint Guided Symbolic Execution. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.
- [81] David Trabish, Shachar Itzhaky, and Noam Rinetzký. 2021. Address-aware query caching for symbolic execution. In *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 116–126.
- [82] David Trabish, Noam Rinetzký, Sharon Shoham, and Vaibhav Sharma. 2023. State Merging with Quantifiers in Symbolic Execution. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1140–1152.
- [83] Chen Wang, Qingyun Duan, Wei Gong, Aizhong Ye, Zhenhua Di, and Chiyuan Miao. 2014. An evaluation of adaptive surrogate modeling based optimization with two benchmark problems. *Environmental Modelling & Software* 60 (2014), 167–179.
- [84] Xinyu Wang, Jun Sun, Zhenbang Chen, Peixin Zhang, Jingyi Wang, and Yun Lin. 2018. Towards Optimal Concolic Testing. In *Proceedings of the 40th International Conference on Software Engineering (ICSE '18)*. 291–302.
- [85] Zhongyi Wang, Linyu Yang, Mingshuai Chen, Yixuan Bu, Zhiyang Li, Qiuye Wang, Shengchao Qin, Xiao Yi, and Jianwei Yin. 2024. Parf: Adaptive parameter refining for abstract interpretation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1082–1093.
- [86] Mark Wetzlinger, Adrian Kulmburg, and Matthias Althoff. 2021. Adaptive parameter tuning for reachability analysis of nonlinear systems. In *Proceedings of the 24th International Conference on Hybrid Systems: Computation and Control*. 1–11.
- [87] E. Wong, L. Zhang, S. Wang, T. Liu, and L. Tan. 2015. DASE: Document-Assisted Symbolic Execution for Improving Automated Software Testing. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering (ICSE '15)*. 620–631.
- [88] Jia Wu, Xiu-Yun Chen, Hao Zhang, Li-Dong Xiong, Hang Lei, and Si-Hao Deng. 2019. Hyperparameter optimization for machine learning models based on Bayesian optimization. *Journal of Electronic Science and Technology* 17, 1 (2019), 26–40.
- [89] Qiuping Yi, Zijiang Yang, Shengjian Guo, Chao Wang, Jian Liu, and Chen Zhao. 2015. Postconditioned Symbolic Execution. In *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST '15)*. 1–10.
- [90] Qiuping Yi, Zijiang Yang, Shengjian Guo, Chao Wang, Jian Liu, and Chen Zhao. 2018. Eliminating Path Redundancy via Postconditioned Symbolic Execution. *IEEE Transactions on Software Engineering* (2018), 25–43.
- [91] Qiuping Yi, Yifan Yu, and Guowei Yang. 2024. Compatible Branch Coverage Driven Symbolic Execution for Efficient Bug Finding. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 1633–1655.
- [92] Shin Yoo and Mark Harman. 2010. Using hybrid algorithm for pareto efficient multi-objective test suite minimisation. *Journal of Systems and Software* 83, 4 (2010), 689–701.
- [93] Jaeha Yoon and Sooyoung Cha. 2024. FeatMaker: Automated Feature Engineering for Search Strategy of Symbolic Execution. *Proc. ACM Softw. Eng. FSE* (2024).